

TCMP 用例管理与执行平台 · 设计思路

项目	内容
文档名称	TCMP 平台设计思路 (Design Overview)
版本	v1.0
编写日期	2026-07-06
关联文档	用例管理和执行系统需求规格书.md、README.md

1. 平台定位

TCMP (Test Case Management & Execution Platform) 解决公司当前测试工作的三个核心痛点：

- 用例散落**：用例分散在 Excel / Word / 个人电脑，缺乏统一资产管理。
- 执行汇总慢**：每轮回归结果靠人工整理，耗时长且易漏。
- 缺陷无溯源**：缺陷与用例、版本无关联，多轮回归无法横向对比。

V1 上线 3 个月内的量化目标：

KPI	目标
用例资产数字化率	≥ 90%
单轮次执行汇总耗时	4h → < 10min
缺陷与用例关联率	≥ 95%

2. 核心设计思想

2.1 三层数据模型解耦资产与执行

平台最关键的设计决策：**不把"用例"与"项目/轮次"耦合**，而是划分为三层，让资产管理、项目组织、执行记录三件事互不污染。

用例集 (Case Set) → 资产层：企业级用例库，与项目无关
↓ 按需挑选
项目 (Project) 用例池 → 组织层：项目按需引用用例（不复制）
↓ 布尔筛选 + 多人分配
测试轮次 (Round) 用例实例 → 执行层：每轮独立结果、执行人、缺陷链接

收益：

- 同一用例可在多个项目、多个轮次里被并行执行，历史独立可追。
- 用例集是"资产"，项目池只存"引用 + 快照版本号"，轮次实例是"执行副本"——三种生命周期分开管理。
- 在执行页临时修订用例时，写回用例集并 `version+1`，旧版本快照通过 `case-version` 表保留，保证历史轮次仍能看到当时执行的那一版。

2.2 执行闭环：Fail → 缺陷 → 报告

失败即建单，用例—缺陷—版本天然联动：

- 双栏执行页 + `P / F / B / N / T` 快捷键批量打结果。
- 选 **Fail** 立即弹出建缺陷对话框，自动回填测试步骤、用例版本、用例编号。
- 通过适配器写到 Teambition，回写 `defect-link` 表建立关联。
- 关闭轮次一键生成报告：模块充分性表 + 缺陷汇总，PRD \$FR-10 三档配色导出 Excel。

2.3 集成用适配器隔离

所有外部系统（钉钉、Teambition、SMTP）都封装在 `backend/src/integrations/` 下的适配器接口后：

- MVP 全部是 Mock 实现（写库 + 控制台日志），研发不依赖外部环境即可跑通全链路。
- 替换真实集成时，只改 `integrations.module.ts` 里的 `useClass`，业务层零改动。

2.4 零依赖起步、平滑演进

- 后端选 SQLite + TypeORM：单文件持久化，部署不依赖外部数据库。
- 实体拆分较细（`case-version` / `defect-link` / `dingtalk-push-log` / `round-case-instance` 独立表），未来平滑迁 Postgres。
- 前端 Vue3 + Vite + Element Plus + Pinia + ECharts，标准组合，学习成本低。

3. 业务流程

3.1 用例资产 → 项目 → 轮次 → 执行 → 报告

[1] 导入 Excel / 在线编辑
↓
用例集 (4 级模块树 + 版本快照)
↓ 项目挑选
[2] 项目用例池 (引用 + 版本号)
↓ 4 步向导
[3] 轮次: 布尔筛选 → 分配 → 发布 → 钉钉推送
↓
[4] 执行页 (快捷键 P/F/B/N/T + Fail 联动建缺陷)
↓
[5] 关闭轮次 → 报告 + 缺陷看板

3.2 轮次 4 步向导

1. **基本信息**: 轮次名、版本号、执行周期。
2. **筛选策略**: 模块 / 等级 / 标签 / 用例类型的布尔组合, 实时预览命中数。
3. **分配**: 勾选执行人, 可按用例平均分派或整轮分派。
4. **确认发布**: 生成 `round-case-instance` 实例, 触发钉钉推送。

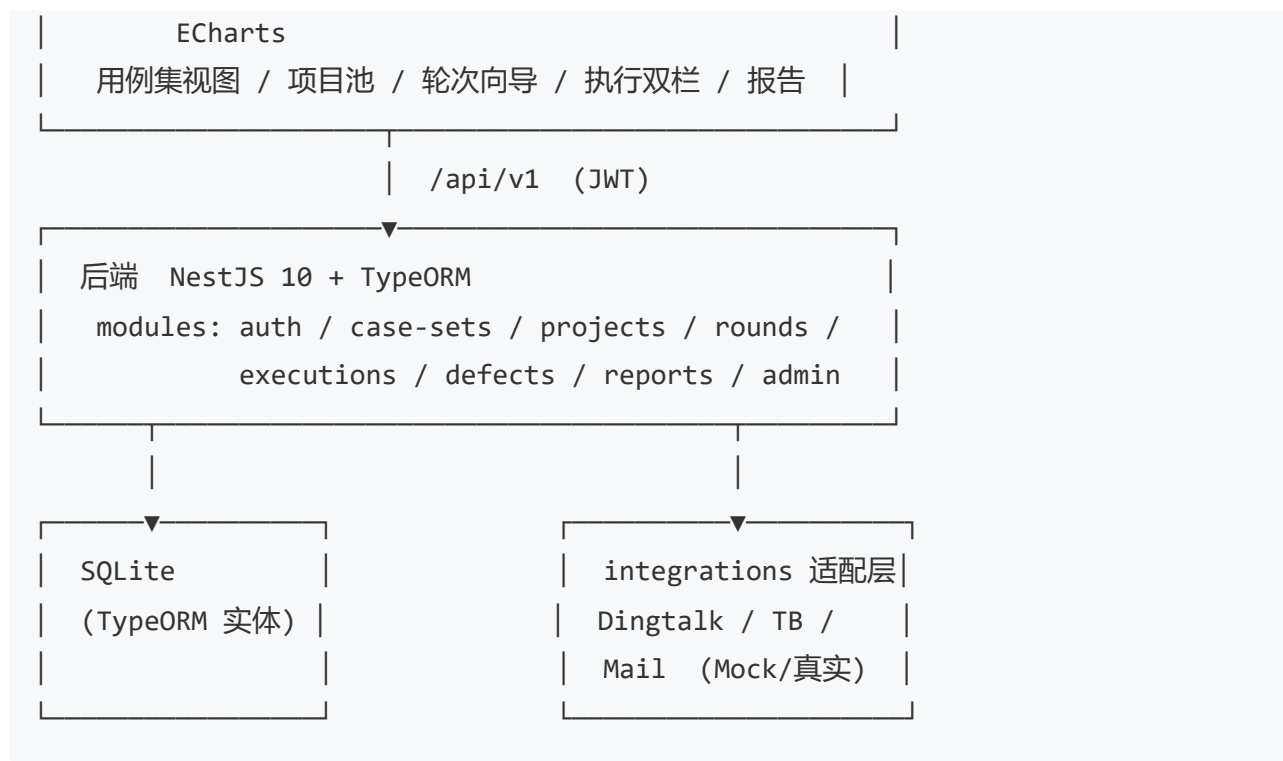
3.3 执行页与快捷键

- 左侧用例树 + 状态指示, 右侧执行详情表单。
- 快捷键: `P` Pass · `F` Fail · `B` Block · `N` N/A · `T` To-Retest。
- 打 F 后必填"实际结果", 直接弹出建缺陷对话框; 缺陷提交后 `defect-link` 建立回链。
- 允许在执行页就地修订用例文本, 写回用例集并生成新版本。

4. 技术架构

4.1 分层结构

前端 Vue 3 + TS + Vite + Element Plus + Pinia +



4.2 主要数据实体

实体	作用
user / user-project-role	系统用户 + 项目级 RBAC
case-set / module / case-set-case	用例集与 4 级模块树
case-version	用例版本快照，支持历史追溯
project / project-case-ref	项目及其用例池引用
round / round-case-instance	测试轮次与用例执行实例
defect-link	用例 ↔ 缺陷 ↔ TB 任务链接
attachment / audit-log	附件与操作审计
dingtalk-push-log	钉钉推送日志 (Mock 也落库)

4.3 后端模块划分 (backend/src/modules/)

- **auth**: 邮箱白名单 + JWT + 5 次错误锁 15 分钟。
- **case-sets**: CRUD + 4 级模块树 + Excel 导入导出 + 版本快照。
- **projects**: 项目 CRUD + 成员管理 + 用例池 + 引用版本刷新。
- **rounds**: 布尔树筛选 + 4 步向导 + 多人分配 + 发布/关闭。
- **executions**: 执行结果录入 + 用例就地修订。
- **defects**: 缺陷建单 + TB 联动 + 项目缺陷看板。

- **reports**: 轮次报告在线视图 + Excel 导出 (PRD 配色)。
- **admin / users / collab / tb-filler / persistence**: 管理、协作、辅助填单、持久化支持模块。

4.4 前端视图划分 (frontend/src/views/)

Login / Register / Admin / CaseSets / Projects / Rounds / Defects , 配合 Layout.vue 侧栏导航。所有 API 请求走 Vite /api 代理, 鉴权通过 Pinia store 统一挂 JWT。

4.5 报告导出配色 (对齐 PRD §FR-10)

区间	颜色	ARGB
≥ 90%	绿	FF92D050
30% – 90%	黄	FFFFFF00
< 30%	红	FFFF0000
表头	浅灰	FFD9D9D9

5. 关键工程约束与经验

在落地过程中沉淀的几条工程约束, 直接写进了代码或部署脚本:

5.1 大批量导入

- NestExpressApplication 的 body limit 放到 50MB (默认 100KB 会让 1000 条 JSON 直接 500) 。
- bulkCreate 改单事务: 一次扫描求各前缀最大编号 → 内存自增分配 (消除每条全表扫描的 O(n²)) ; 模块路径按 path 缓存; caseRepo.save(entities, {chunk: 500}) 分块提交; 版本快照同事务。
- 前端分批提交 (每批 2000) + 进度显示。
- 实测: 万级用例从"不可能"降到 2.3s, 且近线性。

5.2 sql.js 的坑

- 单条 delete/update 也会 autoSave 序列化整库, 避免并发批量单删 (会 ERR_CONNECTION_REFUSED) 。

- 万级逐条删除不要用 sql.js 做，走事务批量。

5.3 乐观锁的正确姿势

- `@VersionColumn` + `repo.save()` 不能可靠触发冲突异常（后写者不报错）。
- 改用显式条件更新：`createQueryBuilder().update().set({... , version: () => 'version + 1'}).where('id=:id AND version=:v')`，检查 `res.affected === 0` 判冲突。
- 前端必须把“打开时读到的 version”一起提交，服务端不能自己重新 `findOne` 拿最新版本再更新，否则退化为 last-write-wins。

5.4 网络与证书

- 公司出口是 sangfor HTTPS 中间人，Node fetch 默认走 bundled CA 会全挂。
- Node 22 加启动参数 `--use-system-ca` 走 Windows 证书库；**不能用** `NODE_TLS_REJECT_UNAUTHORIZED=0` 蒙混。

5.5 前端图表

- ECharts 放在 el-tabs 非激活 tab 时，`onMounted` 时容器 0×0，直接空白。
- 复用 echarts 实例 + 跳过 0×0 容器 + `ResizeObserver` 监听容器尺寸变化时重画并 `resize`。

5.6 构建与部署

- `tsconfig.json` 明确 `"include": ["src/**/*.ts"]` + `"exclude": ["scripts", "dist", "node_modules"]`，避免辅助脚本上移公共 `rootDir` 导致 `dist/main.js` 变成 `dist/src/main.js`，nssm 静默启动失败。

6. 范围划线（V1 明确不做）

在 V1 范围内	不在 V1 范围内
用例集 CRUD、导入、在线编辑	自动化用例脚本执行
项目管理 + 轮次执行	性能测试管理
钉钉任务推送	测试环境管理
缺陷系统（TB）联动建单	测试需求管理

在 V1 范围内	不在 V1 范围内
轮次报告与缺陷看板	测试资源/工时统计
邮箱注册 + 登录 + RBAC	SSO / LDAP

Mock 适配器保留了后续替换真实集成的能力，评审流程、移动端、对象存储、i18n 等留在 PRD §12 开放问题。

7. 一句话总结

用 "用例集 / 项目池 / 轮次实例"三层数据模型 解耦资产与执行；用 适配器层 隔离外部系统；用 NestJS + TypeORM + SQLite 做零依赖起步、平滑演进；把"人工汇总、缺陷无溯源"的痛点用 执行页快捷键 + 自动建单 + 报告一键导出 打成闭环。